

Bài thực hành
Chuyên đề ngôn ngữ lập trình KHMT
Bộ môn KHMT K.CNTT&TT T. Đại học Cần Thơ
Giảng viên: Nguyễn Bá Diệp
Sinh viên thực hiện: Trần Quốc Toàn – B2410699

Phần 1 prolog cơ bản

1) Hợp nhất với "="

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- xe = honda.
no
| ?- xe = xehoi.
no
| ?- honda = xemay.
no
| ?- toyota = honda.
no
| ?- |
```

Kết luận: Trong bốn câu đã cho, tất cả đều trả về **no** vì chúng so sánh các hằng có tên khác nhau và không thể hợp nhất.

2) Hợp nhất atom và biến với vị từ =

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- Xe = honda.
Xe = honda
yes
| ?- honda = Xe.
Xe = honda
yes
| ?- Xe = honda, write(xe).
xe
Xe = honda
yes
| ?- Write(Xe).
uncaught exception: error(syntax_error('user_input:2 (char:40) . or operator expected after expression'),read_term/3)
| ?- Toyota=Xe,Toyota=xe4banh.
Toyota = xe4banh
Xe = xe4banh
yes
| ?- write(Xe),Toyota=xe4banh,Toyota=Xe,write(Xe).
_23xe4banh
Toyota = xe4banh
Xe = xe4banh
(16 ms) yes
| ?- Xe = honda,toyota = Xe.
no
| ?- Xe = honda. Xe = toyota.
Xe = honda
```

```
yes
Xe = toyota
yes
| ?- Xe = honda.
Xe = honda
yes
| ?- Xe = honda.
Xe = honda
yes
| ?- Xe = toyota.
Xe = toyota
(15 ms) yes
| ?- |
```

- Xe là biến, honda là atom. Vì xe chưa có giá trị nên Prolog gán, do đó phép hợp nhất thành công.
- Mặc dù đổi vị trí nhưng phép hợp nhất không phân biệt trái hay phải.
- Thực hiện theo thứ tự Xe nhận giá trị honda, write(xe) do xe là atom nên chỉ in ra xe.
- Lỗi. Vì Write bắt đầu bằng **chữ hoa** nên Prolog coi đó là **biến**. Nhưng ở vị trí đầu của truy vấn, Prolog mong đợi **một vị từ (predicate)** để gọi, không phải một biến.
- Do hai biến liên kết với nhau là Toyota = Xe, Toyota nhận giá trị xe4banh nên Xe cũng bằng xe4banh.
- Đầu tiên Xe chưa có giá trị nên Prolog in _23, sau đó Toyota = xe4banh, Toyota = Xe (câu e) => Xe nhận giá trị xe4banh.
- Đầu tiên thì Xe = honda (Xe được gán giá trị), sau đó toyota = Xe kết quả sẽ ra no vì toyota = honda (2 atom khác nhau nên hợp nhất thất bại).
- Đây là **hai truy vấn riêng**. Sau khi câu đầu kết thúc, biến Xe bị hủy. Sang truy vấn mới, Xe lại là biến mới.

3) Hợp nhất biến (tt)

- Câu lệnh thực hiện được, biến **Computer** được gán giá trị **sun**.
- Câu lệnh thực hiện được, biến **Macintosh** được gán giá trị **great**.
- Câu lệnh thực hiện được, hai biến chưa có giá trị nên Prolog hợp nhất chúng lại. Chúng cùng tham chiếu đến một giá trị, nhưng **chưa có giá trị cụ thể**.
- Câu lệnh thực hiện được, biến **Computer** có giá trị là **sun**, còn biến **Sun** có giá trị **unix**. Không có liên quan gì đến từ sun ở trước vì **sun và unix là hai atom khác nhau**.
- Câu lệnh thực hiện được, **BUGATTI** là biến nên được gán giá trị **champion**.
- Không thực hiện được, giá trị biến không có do tamarrin và marmoset là 2 atom khác nhau nên không thể hợp nhất.

g) Câu lệnh thực hiện được, 3 biến được hợp nhất với nhau.

```
| ?- Computer = sun.  
Computer = sun  
yes  
| ?- Macintosh = great.  
Macintosh = great  
yes  
| ?- BBC = Useful.  
Useful = BBC  
yes  
| ?- Computer = sun, unix = Sun.  
Computer = sun  
Sun = unix  
yes  
| ?- BUGATTI = champion.  
BUGATTI = champion  
yes  
| ?- Monkey = tamarin, Monkey = marmoset.  
no  
| ?- Cathedral = Ely, Ely = Worcester.  
Ely = Cathedral  
Worcester = Cathedral  
yes  
| ?- |
```

4) Kiểm tra giá trị của biến

Ý nghĩa của **var(Term)** và **nonvar(Term)**

- **var(Term)**: Đúng (**yes**) nếu Term là **biến chưa được gán giá trị**.
- **nonvar(Term)**: Đúng (**yes**) nếu Term **không phải là biến chưa gán**, tức là đã được gán hoặc là một atom, số, cấu trúc,...

Sử dụng **var(Term)**

```
| ?- var(Variable).  
yes  
| ?- Car = bugatti, var(Car).  
no  
| ?- var(Car), Car = bugatti.  
Car = bugatti  
yes  
| ?-  
nonvar(unix).  
yes
```

- a) Variable là một biến chưa được gán giá trị nên var() trả về **yes**.
- b) Biến Car đã được gán giá trị bugatti. Lúc này Car không còn là biến chưa gán nữa nên var(Car) trả về **no**.
- c) Ban đầu var(Car) đúng vì Car chưa có giá trị. Sau đó, Car = bugatti (Prolog gán giá trị cho Car). Do cả hai điều kiện đều đúng nên truy vấn thành công.

d) Unix là atom không phải biến, nên nonvar(unix) luôn đúng.

Sử dụng nonvar(Term)

```
?- nonvar(Variable).
no
?- Car = bugatti, nonvar(Car).
Car = bugatti
(16 ms) yes
?- nonvar(Car), Car = bugatti.
no
?- nonvar(unix).
yes
```

- Variable là biến chưa được gán. Do đó **không phải nonvar**, nên kết quả là **no**.
- Sau khi thực hiện Car = bugatti thì Car đã có giá trị nên nonvar(Car) đúng.
- Ngay từ đầu nonvar(Car) Car vẫn là biến chưa được gán nên điều kiện sai. Prolog không thực hiện tiếp và truy vấn thất bại.
- Unix là atom luôn đúng với nonvar.

5) Kiểm tra dữ liệu số các kiểu dữ liệu số sau đây

number(Term)

```
| ?- number(Term).
no
| ?-
  number(1991).
yes
| ?- number(17.5).
yes
| ?- number(haimuoi).
no
| ?- |
```

number(Term) là vị từ dùng để kiểm tra xem Term có phải là một số hay không.

- Nếu Term là **số nguyên hoặc số thực** → yes.
- Nếu Term là **biến chưa gán, atom hoặc cấu trúc** → no.

integer(Term)

```
| ?- integer(Term).  
  
no  
| ?- integer(1991).  
  
yes  
| ?- integer(17.5).  
  
no  
| ?- integer(haimuoi).  
  
no
```

integer(Term) là vị từ dùng để **kiểm tra xem Term có phải là một số nguyên hay không**.

- Nếu Term là **số nguyên** → yes.
- Nếu Term là **số thực, biến chưa gán, atom hoặc cấu trúc** → no.

float(Term)

```
| ?- float(Term).  
  
no  
| ?- float(1991).  
  
no  
| ?- float(17.5).  
  
yes  
| ?- float(chin_phay_ba).  
  
no
```

float(Term) là vị từ dùng để **kiểm tra xem Term có phải là một số thực hay không**.

- Nếu Term là **số nguyên hoặc số thực** → yes.
- Nếu Term là **biến chưa gán, atom hoặc cấu trúc** → no.

6) Kiểm tra dữ liệu

Sử dụng atom(Term)

```
| ?- atom(sqUIrrEl).  
  
yes  
| ?- atom(SQuIRReL).  
  
no  
| ?- atom((debit)).  
  
yes  
| ?- atom('Robert Fitz Ralph (1190-1193)').  
  
yes  
| ?- atom(==>).  
  
yes  
| ?- atom([]).  
  
yes  
| ?- atom([squirrel]).  
  
no
```

atom(Term) kiểm tra Term có phải là **atom (hằng)** hay không.

Sử dụng atomic(Term)

```
| ?- atomic(Term).
no
| ?- atomic(breakfast).
yes
| ?- atom(breakfast), atomic(breakfast).
yes
| ?- number(1991), atomic(1991).
yes
| ?- integer(1991), atomic(1991).
yes
| ?- float(17.5), atomic(17.5).
yes
```

atomic(Term) kiểm tra Term có phải là **dữ liệu nguyên tử (atomic)** hay không.

Bao gồm: atom, số nguyên, số thực

Không bao gồm: biến, danh sách, cấu trúc

7) Biểu diễn các câu sau đây thành dữ liệu sự kiện trong Prolog (có thể phải sử dụng đối số)

```
| ?- [user].
compiling user for byte code...
eat(fred, orange).
eat(fred, dinosaur_meat).
eat(tony, apple).
eat(john, apple).
eat(john, grape).

user compiled, 5 lines read - 573 bytes written, 10036 ms

(78 ms) yes
| ?- eat(fred, orange).
true ?
yes
| ?- eat(john, apple).
true ?
yes
| ?- eat(mike, apple).
no
| ?- eat(fred, apple).
no
```

8) Viết vị từ tuổi để biểu diễn các tri thức sau trong Prolog

```
| ?- [user].
compiling user for byte code...
tuoi(john, 32).
tuoi(agnes, 41).
tuoi(george, 72).
tuoi(ian, 2).
tuoi(thomas, 25).

user compiled, 5 lines read - 552 bytes written, 11924 ms

(78 ms) yes
| ?- tuoi(thomas, X).

X = 25

yes
| ?- tuoi(agnes, 41).

yes
| ?- tuoi(ian, hai).

no
```

ở câu c) 2 là số nguyên, hai là atom. Prolog không hiểu hai là 2 nên cho kết quả là no.

Phần 2: Cơ chế suy luận của Prolog

9) Biểu diễn các tri thức sau thành sự kiện trong Prolog

```
| ?- [user].
compiling user for byte code...
thich(toan, mai).
thich(dung, mai).
thich(mai, hiep).
thich(hiep, dung).
thich(dung, hung).

user compiled, 5 lines read - 551 bytes written, 4937 ms

(94 ms) yes
| ?- thich(X, mai).

X = toan ?

yes
| ?- thich(X, mai).

X = toan ? ;

X = dung ?

(16 ms) yes
| ?- thich(mai, X).

X = hiep

yes
| ?- thich(dung, X).

X = mai ? ;

X = hung

yes
| ?-
thich(dung, X).

X = mai ?

yes
```

10) Cho cơ sở dữ liệu tri thức sau:


```
| ?- [user].
compiling user for byte code...
tape(1,van_morrison,astrol_weeks,ladam_george).
tape(2,beatles,sgt_pepper,a_day_in_the_life).
tape(3,beatles,abbey_road,something).
tape(4,rolling_stones,sticky_fingers,brown_sugar).
tape(5,eagles,hotel_california,new_kid_in_town).

user compiled, 5 lines read - 1030 bytes written, 6676 ms

(109 ms) yes
| ?- tape(5, Artist, Album, Song).

Album = hotel_california
Artist = eagles
Song = new_kid_in_town

yes
| ?- tape(Number, Artist, sgt_pepper, Song).

Artist = beatles
Number = 2
Song = a_day_in_the_life ?

(47 ms) yes
```

11) Định nghĩa luật

```

| ?- [user].
compiling user for byte code...
cool(X) :-
    red(X),
    car(X).

cool(X) :-
    blue(X),
    bike(X).

car(vw_beatle).
car(ford_escort).
bike(harley_davidson).

red(vw_beatle).
red(ford_escort).

blue(harley_davidson).

user compiled, 16 lines read - 1172 bytes written, 4944 ms

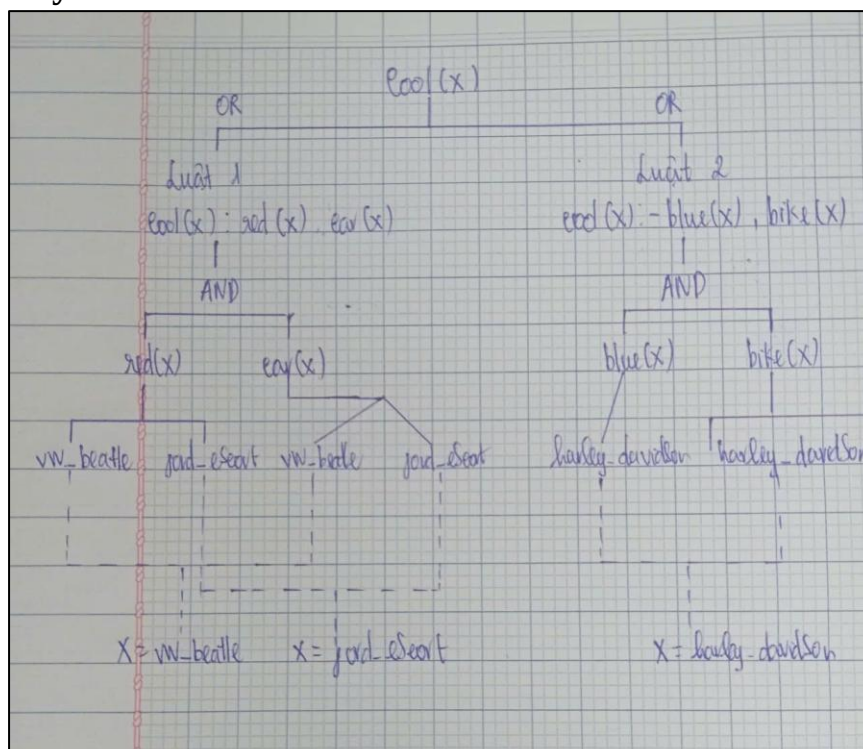
(46 ms) yes
| ?- cool(X).

X = vw_beatle ?
yes
| ?- cool(X).

X = vw_beatle ? ;
X = ford_escort ? ;
X = harley_davidson
yes

```

Cây tìm kiếm



12) Biểu diễn các phát biểu sau bằng Prolog. Chú ý các vị từ giống nhau phải được viết liên tiếp nhau

```
| ?- [user].
compiling user for byte code...
food(chicken).
food(apple).

food(X) :-
    eats(Y, X),
    alive(Y).

eats(bill, peanuts).

eats(john, X) :-
    food(X).

eats(sue, X) :-
    eats(bill, X).

alive(bill).

user compiled, 16 lines read - 932 bytes written, 4354 ms

(63 ms) yes
| ?- food(X).

X = chicken ?

yes
| ?- eats(john, peanuts).

true ?

(16 ms) yes
| ?- eats(sue, X).

X = peanuts

yes
```

```
| ?- eats(X, Y).

X = bill
Y = peanuts ? ;

X = john
Y = chicken ? ;

X = john
Y = apple ? ;

X = john
Y = peanuts ?

(16 ms) yes
```

40) Áp dụng khung chương trình tìm kiếm theo chiều rộng hoặc sâu, giải bài toán 3 tu sĩ và 3 con quỷ.

```
| ?- [user].
compiling user for byte code...
% Trang thái: state(MLeft,CLeft,Boat)
% Boat = left ho?c right

start(state(3,3,left)).
goal(state(0,0,right)).

% Các phép di chuyển?n
move(state(M,C,left), state(M2,C2,right)) :-
    member((DM,DC), [(2,0),(0,2),(1,1),(1,0),(0,1)]),
    M2 is M-DM,
    C2 is C-DC,
    valid(state(M2,C2,right)).

move(state(M,C,right), state(M2,C2,left)) :-
    member((DM,DC), [(2,0),(0,2),(1,1),(1,0),(0,1)]),
    M2 is M+DM,
    C2 is C+DC,
    valid(state(M2,C2,left)).

% Kiểm tra trạng thái hợp lệ?
valid(state(M,C,_)) :-
    M>=0, M<=3,
    C>=0, C<=3,
    MR is 3-M,
    CR is 3-C,
    (M=0 ; M>=C),
    (MR=0 ; MR>=CR).

% DFS
dfs(State,Goal,Visited,[Goal]) :-
    State = Goal.
user:30-31: warning: singleton variables [Visited] for dfs/4

dfs(State,Goal,Visited,[State|Path]) :-
    move(State,Next),
    \+ member(Next,Visited),
    dfs(Next,Goal,[Next|Visited],Path).

% Chạy chương trình
solve :-
    start(Start),
    goal(Goal),
    dfs(Start,Goal,[Start],Path),
    write(Path), nl.

user compiled, 43 lines read - 6244 bytes written, 13680 ms
(187 ms) yes
| ?- solve.
[state(3,3,left),state(3,1,right),state(3,2,left),state(3,0,right),state(3,1,left),state(1,1,right),state(2,2,left),state(0,2,right),state(0,3,left),s
true ?
(109 ms) yes
```

Bài toán đồng nước

```
| ?- [user].
compiling user for byte code...
% Trạng thái: state(X,Y)
% X: lu?ng nu?c trong thùng 9L
% Y: lu?ng nu?c trong thùng 4L

start(state(0,0)).
goal(state(6,_)).

% Đ? d?y thùng 9L
move(state(_,Y), state(9,Y)).

% Đ? d?y thùng 4L
move(state(X,_), state(X,4)).

% Đ? b? thùng 9L
move(state(_,Y), state(0,Y)).

% Đ? b? thùng 4L
move(state(X,_), state(X,0)).

% Rót t? 9L sang 4L
move(state(X,Y), state(X1,Y1)) :-
    T is min(X,4-Y),
    T > 0,
    X1 is X-T,
    Y1 is Y+T.

% Rót t? 4L sang 9L
move(state(X,Y), state(X1,Y1)) :-
    T is min(Y,9-X),
    T > 0,
    X1 is X+T,
    Y1 is Y-T.

% DFS
dfs(State,Goal,_,[State]) :-
    Goal = state(6,_),
    State = state(6,_).

dfs(State,Goal,Visited,[State|Path]) :-
    move(State,Next),
    \+ member(Next,Visited),
    dfs(Next,Goal,[Next|Visited],Path).

% Ch?y
solve :-
    start(Start),
    goal(Goal),
    dfs(Start,Goal,[Start],Path),
    write(Path), nl.

user compiled, 49 lines read - 4307 bytes written, 8761 ms

(94 ms) yes
| ?- solve.
[state(0,0),state(9,0),state(9,4),state(0,4),state(4,0),state(4,4),state(8,0),state(8,4),state(9,3),state(0,3),state(3,0),state(3,4),state(7,0),state(
true ?
```

== Hết ==